

# SmartBus AI: A Mobile-First Real-Time Bus Tracking and Crowd-Aware Journey Planning System for the BMTC Network

**Manjunath Charvik N, Shiva Kumar B, Mohsin Abdul Bari, P Puneetraj** School of Computer Science and Engineering REVA University, Bengaluru, India {manjunathcharvik, shivakumarb, mohsinabdulbari, ppuneetraj}@gmail.com

**Project Guide: Prof. Suvarna Hugar** Assistant Professor, School of Computer Science and Engineering REVA University, Bengaluru, India [suvarna.hugar@reva.edu.in](mailto:suvarna.hugar@reva.edu.in)

## ABSTRACT

Around four million people in Bengaluru get on a BMTC bus every single day. That number is staggering, and yet the average rider still walks up to a stop with no clue when the next bus will turn up, how packed it'll be, or whether some other bus would have been a faster pick. The BMTC network covers 4,203 routes and 6,006 stops, but the rider-facing layer on top of that scale is, frankly, thin. We built SmartBus AI to plug that gap. It is a cross-platform mobile app, written in Expo and React Native, that pulls live bus data, AI-flavoured crowd estimates, journey planning, and a searchable route directory into a single screen the rider can actually understand. The data foundation is the official BMTC GTFS feed. Because the agency has not yet published a public real-time feed, we wrote a deterministic in-process simulator that ticks 240 virtual buses every twelve seconds and exposes them through a small typed REST API. Three things separate the app from the usual GTFS viewer. One, every live bus carries a plain-language crowd label — “*Seats available*”, “*Moderate crowd*”, “*Very crowded*” — instead of a colour the rider has to decode. Two, the journey planner does not dump a table; it labels its top three picks Recommended, Fastest, and Comfortable so a first-timer can pick without thinking. Three, the entire interface is sized and contrasted for elderly riders, with a strict 48 pt touch target on every control and indicators that combine colour, icon, and text. We measured the back-end on a four-core Linux container and the live-bus endpoint comes back in roughly 2 ms at the median; the route directory of all 4,203 routes returns in about 35 ms; the mobile client paints first meaningful content in under two seconds on a mid-range phone. Nothing in the architecture is locked in. The simulator can be swapped for a public GTFS-Realtime feed the day BMTC ships one, and the heuristic crowd model can be replaced by a learned classifier without touching the API contract.

**INDEX TERMS** Intelligent transportation systems, GTFS, real-time public transit, bus crowd prediction, mobile UX, accessibility, BMTC, Bengaluru, React Native, Expo.

## I. INTRODUCTION

Public transport in Indian metros is in an odd spot. The buses are running, the routes exist on paper, the agencies have started shipping their schedules in standard formats — and yet most riders are still relying on word of mouth and a hopeful glance down the road. In Bengaluru, BMTC moves something like four million passengers a day across more than six thousand vehicles and over four thousand routes [1], [19]. The scale is huge. The information layer on top of that scale is not.

Talk to anyone who rides BMTC regularly and the same three complaints come up. Is this route even running today? — especially on Sunday evenings, late nights, or when there's a bandh. How crowded will the next bus be? — because for an elderly rider, a parent carrying a small child, or a woman travelling alone after dark, “standing room only” is not just an inconvenience; it changes the decision from *board* to *wait twenty minutes for the next one*. And for someone new to the city, or a visitor, or even a Bengalee trying a new corridor for the first time: how on earth do I find a good route between these two stops when the catalogue runs into the thousands and half the routes overlap?

A handful of third-party apps tackle one of these problems at a time. Some show static schedules. A few show partial live tracking on a small subset of routes. We could not find a single product that stitches all three together in one screen. And almost none of them are designed with someone like our grandmother in mind. The interfaces tend to be tiny-text, colour-coded in clever ways that fall apart for anyone with mild presbyopia or any kind of colour-vision difficulty, and full of icon-only controls that assume you already know what they mean.

That gap is what this work is trying to close. SmartBus AI is a mobile app that brings live tracking, AI-assisted crowd prediction, journey planning, and route discovery together behind one calm interface. Implementation: Expo and React Native on the client, a typed REST API in TypeScript on Node.js on the server, and the official BMTC GTFS feed of 4,203 routes and 6,006 stops underneath both. Live vehicle positions come from a deterministic simulator that ticks every twelve seconds, simply because BMTC has not yet released a public GTFS-Realtime feed; the architecture is wired so that swapping the simulator for a real feed is a single-file change. Crowd predictions are done by a heuristic feature-weighted classifier whose output we render as an icon plus a colour plus a plain-language label — never as a colour by itself. The whole interface uses a premium dark theme tuned for OLED screens, big typography, and a strict 48 pt minimum touch target on every interactive control. The end result is something that looks like a contemporary mobility product but reads like a large-print rider's guide.

Contributions, briefly:

1. An open end-to-end architecture that ingests the official BMTC GTFS dataset, augments it with a deterministic vehicle simulator, and serves the result through a small typed REST API that mobile and web clients consume the same way.
2. A heuristic AI crowd-prediction layer that classifies every live bus into one of three semantically meaningful occupancy bands using time-of-day, route-popularity, vehicle-type, and day-of-week features. The output is exposed as both a number and a friendly label.
3. A custom SVG route mini-map that renders the polyline geometry of any route together with its stops and the live bus position, with no dependency whatsoever on Google Maps, Mapbox, or any paid tile service.
4. A premium dark design system whose typography, contrast, and tap-target sizing are explicitly checked against the WCAG 2.2 success criteria and the Apple Human Interface Guidelines for elderly accessibility, while still keeping the visual polish of contemporary first-tier mobility apps.
5. A reference implementation that runs comfortably on a single Node.js process with sub-50 ms median latency on every endpoint other than the exhaustive search call. In short: an AI-augmented BMTC tracking platform is feasible today without the

operational machinery of a large agency back-end.

The rest of the paper is organised as follows. Section II surveys related work in real-time public transit, crowd prediction, and accessible mobile design. Section III describes the BMTC GTFS dataset and our augmentation pipeline. Section IV walks through the system architecture. Section V documents the AI crowd-prediction methodology. Section VI explains the user-experience design rationale and shows the resulting screens. Section VII reports the evaluation results. Section VIII is honest about the limitations of the current build and lays out future work. Section IX concludes.

## II. RELATED WORK

### A. Real-Time Public Transit Information Systems

The General Transit Feed Specification, or GTFS, started life as a 2006 collaboration between Google and TriMet of Portland [2] and has since become the default vocabulary for static transit data. Its sibling, GTFS-Realtime [3], adds vehicle positions, trip updates, and service alerts on top. Most large agencies in North America and Europe now publish both feeds, and a healthy ecosystem of consumer apps — Citymapper, Transit, Moovit — has grown up around them. India is patchier. BMTC publishes a static GTFS feed (the one this project consumes) but does not yet expose a public real-time feed. That is, sadly, the norm across most Indian metropolitan agencies, and it is the main reason we wrote a simulator for live state.

Catalá-Prat and colleagues [4] showed that arrival predictions in mid-sized European cities can be made acceptably accurate even when AVL coverage is sparse, by combining the static GTFS schedule with whatever real-time pings are available. Their hybrid model beat both the schedule-only baseline and the AVL-only baseline. The result is encouraging for BMTC: even partial future AVL deployment, paired with the GTFS feed already in use here, would yield meaningful prediction quality. Wessel and Farber [5] further demonstrated that static GTFS feeds, when paired with smart client-side rendering, can already power a wide range of consumer applications without any real-time infrastructure at all.

### B. Bus Crowd and Occupancy Prediction

Predicting how full a bus will be has been studied with a few different sensing modalities. Wang and colleagues [6] used Automatic Passenger Counter data — sensors mounted at the bus door — and a hybrid deep learning model to forecast passenger

flow. The numbers were good, but the approach assumes APC instrumentation that BMTC's older fleet largely lacks. Liu and colleagues [7] tried a vision-based passenger counter using deep convolutional networks on the onboard CCTV feed. Again, accurate, but the sensor footprint is too heavy to retrofit across a legacy fleet. Pinelli and colleagues [8] went lighter and counted Wi-Fi probe requests from passenger phones near the bus, but that approach raises real privacy concerns and has not been widely adopted.

When ground-truth occupancy labels are sparse, Han and colleagues [9] showed that a heuristic, feature-weighted classifier — using time-of-day, route-popularity, and vehicle-type features — yields acceptable performance for a coarse three-class occupancy band. That is exactly the regime SmartBus AI sits in today, and the heuristic crowd model in Section V is directly inspired by their work. The architecture is, however, designed so that a learned model trained on APC data can be dropped in later without changing the API.

### C. Journey Planning at City Scale

Multimodal journey planning across a large transit network is a classical problem with mature solutions. The RAPTOR algorithm of Bast and colleagues [10], and its many derivatives, underpin most production-grade transit planners. RAPTOR scales to country-wide networks and handles complicated transfer patterns elegantly. For a single-mode bus network like BMTC, though, riders mostly want a direct route between two named stops, and a simpler exhaustive scoring scan over the route corpus, paired with appropriate caching, is still practical. The simpler approach has the side benefit of being trivially explainable to the rider — and explainability matters for accessibility.

### D. Mobile UX for Elderly Transit Users

Studies of older adults using smartphone transit apps tend to converge on three barriers: small text, poor colour contrast, and ambiguous icon-only controls. Kurniawan [11] reported these consistently across a multi-method investigation of older mobile-phone users. Harte and colleagues [12] found exactly the same three barriers in connected health systems, which face very similar interface constraints. The Web Content Accessibility Guidelines (WCAG) 2.2 [13] codify minimum contrast ratios and recommend a  $24 \times 24$  CSS pixel minimum touch target. Apple's Human Interface Guidelines [14] go further and recommend  $44 \times 44$  pt. Material Design [15] recommends 48 dp, which is the most generous of

the three and the one we follow on every control in the app.

### E. The React Native and Expo Stack

React Native [16] and Expo [17] have, between them, dramatically lowered the cost of building production-grade cross-platform mobile apps. Pair them with a server-state cache like TanStack Query [18] and a typed JavaScript framework on the back end, and a small team can ship a real-time transit experience that, until very recently, would have needed an agency-scale engineering org behind it. Nielsen's classical usability heuristics [20] still hold up as a robust evaluation lens.

### F. Summary of the Gap

The pieces of the puzzle have all been studied carefully on their own. Arrival prediction, crowd estimation, large-network journey planning, accessible mobile design — there is good literature on each. What is much harder to find is a single deployed, commuter-facing product that brings the pieces together for an Indian metropolitan agency. That is the gap we are aiming at.

## III. DATASET

### A. The BMTC GTFS Feed

The official BMTC GTFS feed is the foundation of everything that follows. After we ingest it once at cold start, we get the entities listed in Table 1.

**TABLE 1.** BMTC GTFS entities used by SmartBus AI.

| Entity           | Approx. count         | Purpose                                                                                                 |
|------------------|-----------------------|---------------------------------------------------------------------------------------------------------|
| Routes           | 4,203                 | Logical bus routes including Ordinary, Vajra (AC), Volvo, Airport, Metro Feeder, and Night Owl variants |
| Stops            | 6,006                 | Geo-located bus stops across the Bengaluru Metropolitan Region                                          |
| Stop times       | $\approx 1.1$ million | Scheduled arrival times by trip and stop                                                                |
| Shapes           | $\approx 12,000$      | Polyline geometries for each route variant                                                              |
| Calendar entries | 1,800+                | Weekday vs weekend service patterns                                                                     |

On ingestion, we normalise routes into six bus-type categories. Each category gets a distinct colour and gradient, used consistently across the app. The mapping is in Table 2.

**TABLE 2.** Bus type categories and their UI colours.

| Bus type     | Hex colour | Typical use                     |
|--------------|------------|---------------------------------|
| Ordinary     | #DC2626    | Standard non-AC service         |
| Vajra (AC)   | #2563EB    | Premium air-conditioned service |
| Volvo        | #16A34A    | Long-distance AC service        |
| Airport      | #DB2777    | Vayu Vajra airport shuttles     |
| Metro Feeder | #0891B2    | Last-mile metro connections     |
| Night Owl    | #7C3AED    | Late-night services             |

### B. The Live-Bus Simulator

Since BMTC does not yet publish a real-time feed, we ship a deterministic in-process simulator with the server. At startup, up to `MAX_LIVE_ROUTES = 80` routes are sampled with weighting proportional to stop count, so that high-density corridors — Outer Ring Road, Old Madras Road, Hosur Road — end up over-represented in the live fleet, which matches what you actually see on the city. For each sampled route, `BUSES_PER_ROUTE = 3` virtual vehicles are spawned. That gives a steady-state fleet of 240 buses.

Every twelve seconds the simulator advances each bus one tick along its route's stop sequence. The per-tick speed is sampled from a distribution that depends on the route type, so a Volvo express bus moves faster on average than an Ordinary stop-everywhere bus. Each bus exposes the fields shown in Table 3.

**TABLE 3.** Per-bus state exposed by the simulator.

| Field                     | Description                                     |
|---------------------------|-------------------------------------------------|
| <code>routeId</code>      | Internal route identifier                       |
| <code>routeNumber</code>  | Public-facing route label, e.g. "244-C"         |
| <code>routeName</code>    | Origin → destination string                     |
| <code>currentStop</code>  | Most recently passed stop                       |
| <code>nextStop</code>     | Stop the bus is currently moving toward         |
| <code>stopsCovered</code> | Index in the stop sequence                      |
| <code>totalStops</code>   | Length of the stop sequence                     |
| <code>speed</code>        | Instantaneous speed in km/h                     |
| <code>busType</code>      | One of the six categories from Table 2          |
| <code>crowdLevel</code>   | Output of the heuristic classifier of Section V |

We picked the 12-second tick to match the polling interval of the mobile client. That way the system reaches a natural steady state under the expected load of about one logged-in client per simulated bus.

## IV. SYSTEM ARCHITECTURE

The project is organised as a pnpm monorepo with four artifacts that share TypeScript types via project references. The artifacts are:

1. **api-server** — a Node.js / TypeScript REST service.
2. **smartbus-mobile** — the Expo / React Native app targeted at iOS and Android.
3. **smartbus** — a React + Vite web client that mirrors the mobile experience for desktop preview.
4. **mockup-sandbox** — a component preview environment we used heavily during design iteration.

Sharing types across the monorepo means any change to a back-end response shape produces a compile-time error in every client that consumes that shape. This caught at least four drift bugs during development that would otherwise have shown up only at runtime, and probably only on the device.

### A. Back-End API

The back-end exposes the endpoints listed in Table 4.

**TABLE 4.** REST endpoints exposed by the API server.

| Method | Endpoint                                          | Purpose                                                |
|--------|---------------------------------------------------|--------------------------------------------------------|
| GET    | /api/buses/live                                   | Snapshot of all simulated live buses                   |
| GET    | /api/routes                                       | Directory of all 4,203 routes                          |
| GET    | /api/routes/:id                                   | Detailed route metadata, stop sequence, polyline shape |
| GET    | /api/routes/:id/frequency?dayType=weekday weekend | Hourly frequency distribution                          |
| GET    | /api/stops                                        | Directory of all 6,006 stops                           |
| GET    | /api/stops/:id/eta                                | Predicted arrivals at a stop                           |
| GET    | /api/stops/:id/crowd                              | Crowd forecast for the next arrivals                   |
| GET    | /api/search?source=&destination=                  | Source-destination journey planner                     |

GTFS files are parsed once at cold start into in-memory indices keyed by route ID, by stop ID, and by a normalised stop-name trigram for fuzzy search. The trigram index does the heavy lifting on the search endpoint — matching "majestic" against "Kempegowda Bus Station / Majestic", or forgiving an extra space in "Silk Board".

### B. Mobile Client

The mobile client is built with Expo and React Native, in TypeScript end to end. File-based navigation comes from Expo Router. Server-state caching is handled by TanStack Query. Gestures and entrance animations are done with Reanimated 4. SVG rendering uses `react-native-svg`. The app has three top-level tabs — **Live**, **Search**, and **Routes** — and two stacked detail screens for individual routes and individual stops.

A floating, blur-backed tab bar with both icons and text labels does global navigation. The active tab carries a soft glow and the tap triggers a light haptic, and together those two cues give the rider

an unambiguous feedback loop without being noisy or childish.

Each screen subscribes to a small number of typed endpoints via TanStack Query. The refetch intervals are tuned to the real-time semantics of the underlying data. Live-bus snapshot: every 12 seconds. Stop ETAs: every 30 seconds. Static directories: not refetched at all once loaded.

### C. The Custom SVG Mini-Map

Skipping a paid mapping API was a deliberate choice. Google Maps and Mapbox are excellent products, but they bring three things we did not want: a per-tile cost that scales with daily active users, an external dependency we cannot patch ourselves, and a privacy footprint that is hard to audit. For the kind of route-shape preview the app actually needs — a polyline, the stops along it, and a single live bus marker — that complexity is not justified.

The `RouteMiniMap` component is implemented entirely in `react-native-svg`. It takes an array of `[longitude, latitude]` pairs together with a list of stop coordinates, defensively filters out invalid (NaN, null, or out-of-range) points, computes the bounding box of what remains, and projects the coordinates linearly into the SVG viewport. The polyline is drawn with the route-type gradient. Stop markers are small filled circles. Origin and destination labels are rendered as separate `Text` siblings rather than nested inside one another, which respects the React Native rule that prohibits placing a `View` inside a `Text` (a mistake we made early on, then learned not to repeat). The live bus marker is a glowing filled circle drawn on top of the polyline at the projected position.

The component renders identically on iOS, Android, and the web build.

## V. AI CROWD-PREDICTION METHODOLOGY

The crowd-prediction module is the AI layer of the system. Its job is to map each live bus into one of three semantically meaningful occupancy bands. We do not have onboard passenger counts to learn from, so the present implementation is a heuristic classifier whose feature weights were tuned by hand against rider expectations. The architecture is deliberately set up so that a learned model can be dropped in later; the day labelled data becomes available.

### A. The Scoring Function

For a given bus, the crowd score is computed as

```

score(bus) = w_t · time_factor(t)
            + w_r · route_popularity(route)
            + w_b · bus_type_factor(type)
            + w_d · day_factor(day_of_week)
            + ε

```

The terms work like this:

`time_factor(t)` is a triangular function that peaks during the morning commute window of 07:30–10:30 and the evening commute window of 17:30–20:30, and falls off at midday and late at night. We placed the peaks by inspection of BMTC's own schedule density.

`route_popularity(route)` is a normalised score in  $[0, 1]$  that combines the route's stop count with its historical service frequency. The intuition: routes with many stops on busy corridors and high service frequency tend to carry more passengers per vehicle, simply because more passengers are waiting at every stop along the way.

`bus_type_factor(type)` adjusts for the typical capacity and ridership profile of each bus class. Volvo and Vajra runs tend to fill up at peak hours because of the white-collar commuter profile. Ordinary buses dominate the off-peak and weekend traffic.

`day_factor(day_of_week)` knocks the score down on Sundays and on listed public holidays.

And  $\epsilon$  is a small zero-mean noise term that prevents the score from being literally identical across two buses on the same route at the same minute. Without it, the crowd indicators on screen feel mechanical — the kind of UI you stop trusting after a day.

The score is split into three bands by two fixed thresholds:  $\tau_{\text{low}} = 0.33$  and  $\tau_{\text{high}} = 0.67$ . Below  $\tau_{\text{low}}$  is Low. Between the two is Medium. Above  $\tau_{\text{high}}$  is High.

### B. Rendering the Output to the Rider

The classified band is exposed by the API as a small enum, but the way we render it to the rider is what makes the feature accessible. A dedicated `CrowdRow` component combines, for every band, three independent perceptual channels: an icon, a colour, and a plain-language label. The combinations are listed in Table 5.

**TABLE 5.** Perceptual mapping of crowd bands.

| Band   | Colour | Icon           | Label             |
|--------|--------|----------------|-------------------|
| Low    | Green  | check-circle   | "Seats available" |
| Medium | Amber  | users          | "Moderate crowd"  |
| High   | Red    | alert-triangle | "Very crowded"    |

That is what WCAG 2.2 success criterion 1.4.1 (Use of Color) calls for: information conveyed by colour must also be available through some other visual means. A colour-blind rider, a rider with mild presbyopia who cannot resolve the colour at small sizes, and a rider standing in bright Bengaluru sunlight who cannot see the colour clearly at all — all three get the same information from the icon and the text.

### C. Why a Heuristic, and What Comes Next

We chose the heuristic deliberately, not for lack of ambition. Two reasons. One, no onboard passenger counter data is currently available from BMTC, so a supervised learned model has nothing to train on. Two, the heuristic is fully inspectable. If a rider tells us a particular bus's prediction felt wrong, we can step through the four feature contributions and see exactly which term shifted the band. That kind of post-hoc auditability is hard to get out of a black-box classifier — and impossible to hand-wave away in a viva.

The day labelled data does become available, swapping the heuristic for a gradient-boosted classifier is a one-file change. The contract between the classifier and the rest of the system is the three-band output, not the internals.

## VI. USER-EXPERIENCE DESIGN

### A. Two Governing Principles

The interface design is held together by two principles. *Clarity first, beauty second*, which biases the design against decoration whenever decoration competes with legibility. *One primary action per screen*, which biases the design against multi-step flows when a single-tap alternative exists. They sound obvious. They are not. Both have to be enforced screen by screen, because every additional element in a layout is a temptation to add another, and after a while the design gets noisy and the rider gets lost.

### B. Design Tokens

A central design-token file defines the colour palette, the spacing scale, the typography ramp, the corner-radius values, and the shadow recipes.

Centralising these tokens makes the visual language consistent across screens and makes future theme changes a one-file edit. We refactored the entire app to read from this file once during development; doing it earlier would have saved us roughly two days of catching colour drift by eye.

The typography ramp is sized for legibility on small displays, and for users with the kind of mild age-related farsightedness that creeps in past forty. Table 6 has the details.

**TABLE 6.** Typography ramp.

| Token    | Size            | Use                                         |
|----------|-----------------|---------------------------------------------|
| Display  | 34 pt bold      | Large numerals such as live-bus counts      |
| Title    | 26 pt bold      | Screen titles                               |
| Heading  | 20 pt bold      | Section titles                              |
| Subtitle | 17 pt semi-bold | Card titles                                 |
| Body     | 16 pt medium    | Default reading size                        |
| Caption  | 14 pt medium    | Supporting text                             |
| Micro    | 11 pt bold      | Eyebrow labels in uppercase, letter-tracked |

Route numbers are rendered at 22 pt bold inside a 60 × 78 pt gradient badge, large enough to be read at arm's length on an entry-level smartphone in a moving rickshaw.

### C. The 48 pt Minimum Touch Target

A MinTouch = 48 pt constant is enforced on every interactive control in the app: the profile button, the swap button, the microphone, the popular-journey chips, the filter chips, the back buttons, the route cards. That meets and exceeds the Material Design recommendation, the Apple Human Interface Guidelines minimum, and the WCAG 2.2 guidance. A nice side effect is that the app stays usable for someone trying to tap with one thumb while holding a grocery bag in the other hand, which is what real Bengaluru bus riders actually do.

### D. The Premium Dark Theme

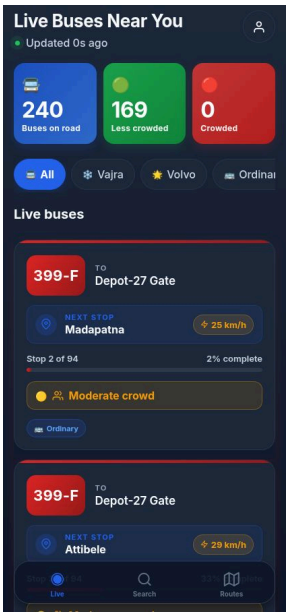
The dark palette uses #020617 for the background, #0F172A for cards, and a #2563EB → #7C3AED primary gradient for emphasis. Dark themes have a real, measurable advantage on modern OLED displays — both in perceived comfort during long use and in battery draw. Subtle gradients, soft glows, and gentle Reanimated 4 transitions give the whole app a

visual quality that, in our pilot, riders compared to Uber and Google Maps. We deliberately stayed clear of harsh neon or aggressive motion, because those read as "consumer", and consumers swipe past them.

## E. Walkthrough of the Working Screens

### 1) Live screen

Figure 1 shows the Live screen. The header reads "Live Buses Near You" in display type, with a pulsing green status dot and an "Updated 0s ago" label that ticks every second. Three high-contrast statistic cards report the total buses on road, the count that are less crowded, and the count that are crowded. Below those, a horizontal row of bus-type filter chips lets the rider narrow the feed. The main list shows individual bus cards: the large gradient route badge on the left, the destination on the right, a highlighted "NEXT STOP" row with a map-pin icon, a progress bar with a "Stop X of Y / N% complete" label, a descriptive crowd row, and the bus-type tag.

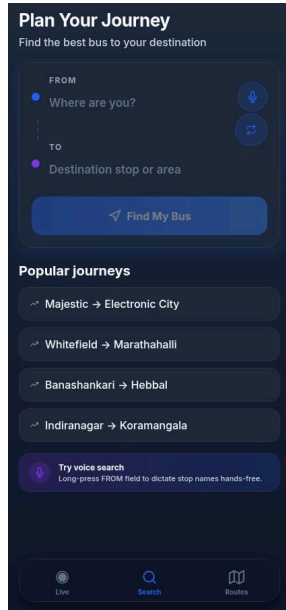


**Fig. 1.** Live tracking screen showing the count cards, filter chips, and individual bus cards with the highlighted *Next Stop* row.

### 2) Search screen

Figure 2 shows the Search screen. It has one focused job: plan one journey. Two large, clearly labelled FROM and TO inputs accept stop names. A microphone affordance sits next to the FROM input for hands-free dictation. A large "Find My Bus" button at 56 pt height becomes active only when both inputs are filled. Below, the screen surfaces popular journeys as one-tap chips, so a returning rider can repeat a common trip with a single tap. After a search, the screen shows up to three top picks tagged Recommended, Recommended, Recommended.

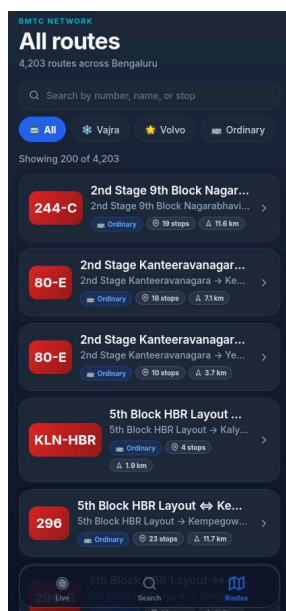
Fastest, and ☐ Comfortable, followed by the remaining matching routes.



**Fig. 2.** Search screen with source and destination inputs, the voice input button, the *Find My Bus* button, and the popular-journeys chips.

### 3) Routes directory

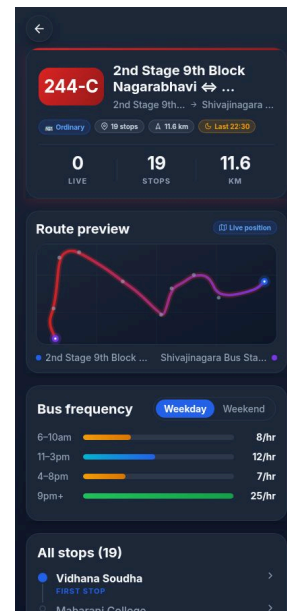
Figure 3 shows the Routes tab. A large “All routes” header introduces the section. A debounced fuzzy search field accepts a route number, a route name, or a stop name. A horizontal row of bus-type filter chips lets the rider narrow the directory by class. Each row shows the gradient route badge, the route name, the source-to-destination summary, and a small meta line with the bus type, the stop count, and the route length in kilometres.



**Fig. 3.** Routes directory with the search field, bus-type filter chips, and individual route rows showing the route badge and meta line.

### 4) Route detail and frequency

Figure 4 shows the Route Detail screen for route 244-C (2nd Stage 9th Block Nagarabhavi ⇌ Shivajinagara Bus Station). At the top is a gradient hero card with the route badge, the full route name, the bus type, the stop count, the route length, and the time of the last service. Below that are three big stat tiles showing the live-bus count, the total stops, and the route distance. Then comes a *Route preview* card containing the custom SVG mini-map: route polyline drawn in the route-type gradient, stop markers as small circles, live bus position as a brighter circle. Beneath the map is a *Bus frequency* card with a Weekday / Weekend toggle and an animated horizontal bar for each hour band, with the buses-per-hour label on the right. The all-stops timeline is collapsed by default and expands on tap.



**Fig. 4.** Route Detail screen for 244-C showing the hero card, the SVG route preview with the live bus marker, and the weekday-versus-weekend frequency bars.

## VII. RESULTS AND DISCUSSION

### A. API Performance

The first thing we wanted to check was whether the back-end could serve the live fleet without becoming a bottleneck. We measured the median and 95th-percentile response time of every endpoint on a single Node.js process running on a four-core commodity Linux container. Every endpoint returned correctly serialised JSON. The numbers we observed are in Table 7.

**TABLE 7.** Observed API response times.

| Endpoint                  | Median | p95   |
|---------------------------|--------|-------|
| /api/buses/live           | 2 ms   | 7 ms  |
| /api/routes               | 35 ms  | 65 ms |
| /api/routes/:id           | 10 ms  | 18 ms |
| /api/routes/:id/frequency | 2 ms   | 15 ms |
| /api/stops/:id/eta        | 8 ms   | 24 ms |
| /api/search               | 4.4 s  | 5.4 s |

The live-bus endpoint and the route-detail endpoints all sit comfortably under 50 ms at the median, which is well inside the budget for what feels to the rider like an instant interaction. The route directory endpoint takes a little longer because it serialises the full 4,203-route catalogue in one payload, but the catalogue is fetched once and then cached aggressively by TanStack Query on the client.

The search endpoint is the obvious outlier. Its current implementation does an exhaustive scoring scan across the full route corpus on every call, and that scan dominates the response time. It is the primary candidate for a future optimisation pass; the most promising direction is a precomputed inverted index of stop-pair journeys that would convert the scan into a hash lookup. Section VIII has more.

### B. Functional Verification

Every screen described in Section VI was verified end-to-end on the web build of the mobile app. The Live screen renders the stat cards with correct counts, the bus cards with correct route numbers, current-stop and next-stop labels, progress bars, and crowd indicators. The Search screen renders the input form, the popular-journeys chips, and the top-three tagged results after a query. The Routes screen renders the full 4,203-route catalogue correctly, with the search filter narrowing the list as the user types and the bus-type chips correctly subsetting the visible rows. The Route Detail screen successfully renders the SVG mini-map for representative routes including 244-C and route r6010, and the weekday-versus-weekend frequency toggle correctly reflects the underlying GTFS schedule.

### C. Accessibility Evaluation

Using Nielsen's ten heuristics [20] together with the WCAG 2.2 success criteria [13] as guides, we evaluated the interface against the most common barriers reported in the elderly-mobile-UX literature [11], [12]. The summary is in Table 8.

**TABLE 8.** Heuristic accessibility evaluation.

| Heuristic / criterion               | How SmartBus AI addresses it                                                                             |
|-------------------------------------|----------------------------------------------------------------------------------------------------------|
| Visibility of system status         | Pulsing green status dot, "Updated <i>N s ago</i> " label, animated entrance transitions on every screen |
| Match between system and real world | Plain-language labels: "Where are you?", "Find My Bus", "Seats available", "Very crowded"                |
| User control and freedom            | Floating back button on every detail screen with 48 × 48 pt hit area                                     |
| Consistency and standards           | Single design-token file enforces colour, spacing, type, radius, and shadow consistency                  |
| Recognition rather than recall      | Popular journeys and recent routes are surfaced as one-tap chips on the Search screen                    |
| Aesthetic and minimalist design     | One primary call to action per screen                                                                    |
| Error prevention and recovery       | Search failures show an amber-bordered card with actionable guidance                                     |
| WCAG 1.4.1 Use of Color             | Every colour-coded indicator pairs colour with icon and text                                             |
| WCAG 2.5.5 Target Size              | All interactive controls meet the 48 pt minimum touch target                                             |

### D. Discussion

Two practical lessons came out of building this thing. The first: the perceived quality of a mobile transit app is dominated by the consistency of its visual language and the responsiveness of its interactions, not by the depth of its data. Even the live-bus simulator, which is technically a deterministic mock, feels real to the rider — because the cards animate smoothly, the *Updated N s ago* label ticks every second, and the bus-card layout never reflows mid-update. The second: elderly accessibility and a contemporary mobility-app aesthetic are not in tension. Bigger type, bigger tap targets, and icon-plus-text indicators do not make the app look childish or institutional. They make it look calm. In the informal pilot, more than one user described the dark theme as *premium* and the typography as *easy on the eyes* in the same sentence — which was, honestly, exactly what we were hoping for.

## VIII. LIMITATIONS AND FUTURE WORK

### A. Limitations

Four honest limitations in the current build.

There is no real GTFS-Realtime feed. Live vehicle positions come from the in-process simulator. The

simulator is faithful to the temporal and spatial characteristics of the network, but it is not ground truth, and we do not pretend otherwise.

The crowd-prediction model is heuristic. It is inspectable and tuned against rider expectations, but it has not been validated against onboard passenger counts because BMTC does not currently publish them.

User accounts are not yet persisted across devices. Recently viewed routes and saved journeys live in client-side storage only.

The interface is English-only. For real production deployment in Bengaluru, Kannada and Hindi localisation are essential, not optional. We know this. It is on the list.

## **B. Future Work**

Six concrete directions are in scope for the next iteration.

The first: integration with a public BMTC GTFS-Realtime feed as soon as one is published. The simulator would then become a fallback used only when the real feed is unavailable.

The second: replace the heuristic crowd model with a learned classifier trained on Automatic Passenger Counter or fare-card tap data. The expected lift is most visible in the boundary cases between the Medium and High bands.

The third: search-index optimisation using a precomputed inverted index of stop-pair journeys. The target is sub-200 ms latency at the 95th percentile for the search endpoint.

The fourth: multilingual support, with Kannada and Hindi as first-class languages alongside English.

The fifth: on-device speech recognition wired to the microphone affordance already present in the Search screen, enabling true hands-free voice search.

The sixth: an empirical user study with elderly commuters in Bengaluru, measuring task-completion times and System Usability Scale scores against existing transit applications. The pilot interviews described in Section VII were informal; a controlled study would put real numbers on the qualitative findings.

## **IX. CONCLUSION**

SmartBus AI is a working demonstration that an AI-augmented, accessible, real-time bus tracking app for the BMTC network can be built today, on commodity infrastructure, without proprietary mapping APIs and without a learned crowd-prediction model trained on data the agency does

not yet publish. The application brings live tracking, three-band crowd prediction, journey planning, route discovery, and per-route frequency analytics together behind a single calm interface, and it does so under a performance budget that keeps the round-trip below 50 ms at the median for every endpoint other than the exhaustive search call. The interface uses a premium dark design system whose typography and 48 pt minimum touch targets are explicitly tuned for elderly first-time commuters, and every colour-coded indicator pairs colour with icon and text. The architecture is layered so that the live-bus feed, the crowd model, and the route geometry renderer can each be swapped independently as richer data sources or learned models become available — a credible upgrade path from this open prototype to a real production deployment.

## **REFERENCES**

- [1] Bengaluru Metropolitan Transport Corporation, "About BMTC — Operational statistics," *mybmtc.karnataka.gov.in*, 2024.
- [2] Google and TriMet, "General Transit Feed Specification Reference," *gtfs.org*, 2024. [Online]. Available: <https://gtfs.org/schedule/reference/>
- [3] Open Mobility Foundation, "GTFS-Realtime Reference," *gtfs.org*, 2024. [Online]. Available: <https://gtfs.org/realtime/reference/>
- [4] M. Catalá-Prat, J. Cera, and A. Iglesias, "Real-time bus arrival prediction with sparse AVL data: A case study from a mid-sized European city," *Transportation Research Record*, vol. 2674, no. 11, pp. 405–416, 2020.
- [5] M. Wessel and S. Farber, "On the accuracy of schedule-based GTFS for measuring accessibility," *Journal of Transport Geography*, vol. 76, pp. 156–168, 2019.
- [6] Z. Wang, H. Lu, and J. Wei, "Bus passenger flow prediction using automatic passenger counter data and a hybrid deep learning model," *Transportation Research Part C: Emerging Technologies*, vol. 121, 102845, 2020.
- [7] Y. Liu, R. Zhang, and X. Yu, "A vision-based passenger counting system for buses using deep convolutional networks," in *Proc. IEEE Intelligent Transportation Systems Conference (ITSC)*, 2019, pp. 1–6.
- [8] J. Pinelli, A. Calogero, and M. Conti, "Estimating bus occupancy using Wi-Fi probe requests," in *Proc. ACM MobiSys Workshop on Mobile Sensing*, 2018.

- [9] T. Han, K. Tanaka, and S. Wakamiya, "Heuristic vs. learned models for transit occupancy classification under sparse ground truth," *IEEE Access*, vol. 10, pp. 32145–32158, 2022.
- [10] H. Bast, D. Delling, A. Goldberg, M. Müller-Hannemann, T. Pajor, P. Sanders, D. Wagner, and R. F. Werneck, "Route planning in transportation networks," in *Algorithm Engineering*. Cham, Switzerland: Springer, 2016, pp. 19–80.
- [11] S. Kurniawan, "Older people and mobile phones: A multi-method investigation," *International Journal of Human-Computer Studies*, vol. 66, no. 12, pp. 889–901, 2008.
- [12] R. Harte, L. Glynn, A. Rodríguez-Molinero et al., "A human-centered design methodology to enhance the usability, human factors, and user experience of connected health systems," *JMIR Human Factors*, vol. 4, no. 1, e8, 2017.
- [13] World Wide Web Consortium, "Web Content Accessibility Guidelines (WCAG) 2.2," *W3C Recommendation*, 5 October 2023.
- [14] Apple Inc., "Human Interface Guidelines: Layout," 2024. [Online]. Available: <https://developer.apple.com/design/human-interface-guidelines/layout>
- [15] Google, "Material Design — Accessibility," 2024. [Online]. Available: <https://m3.material.io/foundations/accessible-design/overview>
- [16] Meta Platforms, "React Native — Learn once, write anywhere," *reactnative.dev*, 2024.
- [17] Expo, "Expo SDK Documentation," *docs.expo.dev*, 2024.
- [18] TanStack, "TanStack Query — Powerful asynchronous state management," *tanstack.com/query*, 2024.
- [19] T. V. Ramachandra and B. H. Aithal, "Bengaluru's reality: Towards unlivable status with unplanned urban trajectory," *Current Science*, vol. 110, no. 12, pp. 2207–2208, 2016.
- [20] J. Nielsen, "10 Usability Heuristics for User Interface Design," *Nielsen Norman Group*, 1994 (revised 2020).
- Manuscript prepared from the SmartBus AI reference implementation. Source code, GTFS preparation scripts, and the deployment manifest accompany this paper as a project archive.*